Accueil du niveau

FICHIERS ET MODULES

Lire, écrire et organiser

FICHIERS ET MODULES

📄 fichier

↔

🐍 Python

1 / 9

Accueil du niveau

LIRE UN FICHIER TEXTE

open, read et readlines

```
# Lire tout le contenu d'un coup
f = open('notes.txt', 'r')
contenu = f.read()
print(contenu)
f.close()

# Lire ligne par ligne dans une liste
f = open('notes.txt', 'r')
lignes = f.readlines()
for ligne in lignes:
    print(ligne.strip())
f.close()
```

LIRE LIGNE PAR LIGNE

📄 fichier

↔

🐍 Python

2 / 9

Accueil du niveau

LE BLOC WITH

Le gardien qui ferme la porte

```
# Le gardien qui ferme la porte derriere vous
with open('notes.txt', 'r') as f:
    contenu = f.read()
    print(contenu)
# Ici, f est automatiquement ferme

# Lire ligne par ligne (economise la memoire)
with open('notes.txt', 'r') as f:
    for ligne in f:
        print(ligne.strip())
```

WITH : GARDIEN AUTOMATIQUE

with : gardien automatique

✨ →

propre

3 / 9

Accueil du niveau

ÉCRIRE DANS UN FICHIER

Modes w et a

```
# Écrire (écrase le fichier existant)
with open('resultat.txt', 'w') as f:
    f.write('Ligne 1\n')
    f.write('Ligne 2\n')

# Ajouter à la fin du fichier
with open('resultat.txt', 'a') as f:
    f.write('Ligne 3 ajoutee\n')

# Ecrire plusieurs lignes d'un coup
lignes = ['Alice: 18\n', 'Bob: 15\n', 'Clara:
20\n']
with open('notes_eleves.txt', 'w') as f:
    f.writelines(lignes)
```

ÉCRIRE DANS UN FICHIER

📄 fichier

↔

🐍 Python

Accueil du niveau

LE MODULE CSV

csv.reader et DictReader

```
import csv

# csv.reader : chaque ligne est une liste
with open('eleves.csv', 'r') as f:
    lecteur = csv.reader(f)
    for ligne in lecteur:
        print(ligne)  # ['Alice', '18',
'Paris']

# csv.DictReader : chaque ligne est un dict
with open('eleves.csv', 'r') as f:
    lecteur = csv.DictReader(f)
    for eleve in lecteur:
        print(eleve['nom'], eleve['note'])
```

DONNÉES TABULAIRES CSV

	col_A	col_B	col_C
0	...	...	...
1	...	...	...
2	...	...	...

Accueil du niveau

LE MODULE JSON

load, dump, loads, dumps

```
import json

# Lire un fichier JSON
with open('config.json', 'r') as f:
    data = json.load(f)
    print(data['nom'])  # accès comme un dict

# Ecrire un fichier JSON
resultat = {'score': 95, 'eleve': 'Alice'}
with open('sortie.json', 'w') as f:
    json.dump(resultat, f, indent=2)

# Depuis/vers une chaine (le 's' = string)
texte = json.dumps(resultat)
obj = json.loads(texte)
```

STRUCTURE JSON

```
"Structure JSON_1" →valeur
"Structure JSON_2" →valeur
"Structure JSON_3" →valeur
```

6 / 9

Accueil du niveau

Accueil du niveau

CHEMINS ET DOSSIERS

os et pathlib

```
import os
from pathlib import Path

# Verifier si un fichier existe
print(os.path.exists('notes.txt'))  #
True/False

# Lister les fichiers d'un dossier
for nom in os.listdir('.'):
    print(nom)

# pathlib : la methode moderne
chemin = Path('data') / 'notes.txt'
print(chemin.exists())
print(chemin.suffix)   # .txt
print(chemin.stem)     # notes

# Creer un dossier s'il n'existe pas
Path('resultats').mkdir(exist_ok=True)
```

ARBORESCENCE FICHIERS

Accueil du niveau

7 / 9

Accueil du niveau

IMPORTER DES MODULES

import, from...import, pip

- import math → math.sqrt(16)
- from math import sqrt → sqrt(16)
- import random as rnd → alias pratique
- pip install matplotlib → installe depuis PyPI
- Vos propres fichiers .py sont des modules

IMPORT : BOÎTE À OUTILS

→

→

Accueil du niveau

GESTION D'ERREURS

try / except / finally

```
try:
    with open("inexistant.txt", "r",
encoding="utf-8") as f:
        contenu = f.read()
except FileNotFoundError:
    print("Fichier introuvable")
finally:
    print("Bloc terminé")
```

- Capturer l'erreur la plus précise possible
- Éviter les `except:` trop larges
- Journaliser les cas d'échec importants

9 / 12

Accueil du niveau

ARCHITECTURE MINIMALE D'UN PACKAGE

Structurer le code pour grandir proprement

```
mon_projet/
  main.py
  src/
    __init__.py
    io_utils.py
    business_rules.py
  tests/
    test_business_rules.py
```

- Séparer I/O, logique métier, exécution
- Préparer les tests dès la base
- Nommage clair des modules

10 / 12

Accueil du niveau

CHECKLIST QUALITÉ FICHIERS/MODULES

- Encodage UTF-8 explicite sur les fichiers texte
- Lecture paresseuse pour gros fichiers
- Exceptions explicites et messages actionnables
- Imports cohérents et sans dépendances circulaires

11 / 12

Accueil du niveau

RÉCAPITULATIF

Fichiers et modules

- ✓ with open() : lecture et écriture sécurisées
- ✓ csv.reader / DictReader pour les tableaux
- ✓ json.load/dump (fichiers) vs loads/dumps (chaînes)
- ✓ pathlib pour les chemins modernes
- ✓ import et pip pour étendre Python

12 / 12

Speaker notes